What is digital system? A piece of hardware that processes signals that have a finite number of discrete values. A system that comprises of hardware and software to process inputs and store the results suitably.

- Digital doesn't mean binary, digital computers use binary numbers

- Analog systems produces signals that can take any value in principle across a continuous range

- Binary Number system: Signed (→ +ve) and unsigned (→ +ve/-ve) → default

  n-bits from x_0 (LSB) to x_{n-1} (MSB), then its value is given by $x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 \cdot 2^1 + x_0 \cdot 2^0$

  range: 0 to $+2^n - 1$

- Addition of unsigned numbers → unsigned
- Subtraction of unsigned numbers → unsigned / signed

- 3 ways to represent signed binary numbers:

(i) Sign magnitude representation

$2^n - 1 \rightarrow$ nos

0 in MSB → +ve no. 0101 → 5

1 in MSB → -ve no. 1110 → -6

limitation: non-uniqueness of representing '0' (0000, 1000 → both represent 0)

symmetry is of interest 1111 → 7 0111 → -7 (extreme number dealing)

(ii) 1's magnitude representation n bits: range: $\in (2^{n-1}, 2^{n-1})$

$2^n - 1 \rightarrow$ nos

+ve: same as binary

-ve: write for +ve and flip the bits

limitation: 2 nos have same 1's complement (-7 and 8 (unsigned)), non-uniqueness to represent 0.

(iii) 2's magnitude representation n-bits: range $= [-2^{n-1}, 2^{n-1})$

$2^n \rightarrow$ no's

+ve: Same as binary

-ve: write binary for +ve, flip each bit then add 1.

+7: 0111 -7: 1001

Zero representation is unique, easy to multiply

Arithmetics: $-5+(-3) = 1011+1101 = 11000 = (-8)$

$45 - 27 = 45 + (-27)$

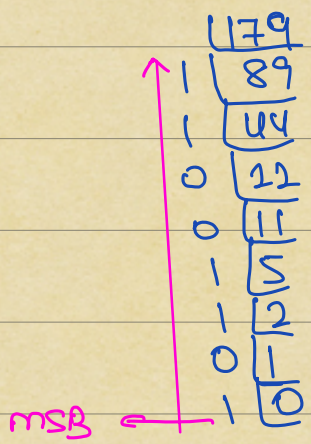
```
 0101101
 1100101
```

$-7.5 = -8+0.5 = 1000.10$

$7.5 - 3 =$

```
 0111.1
 1101.0
-------
 0100.1   → 4.5
```

Decimal to binary:



$01011001_2 = 179_{10}$

For decimal numbers:

$0.6875 \times 2 = 1 + 0.3750$ (a-1)

$0.3750 \times 2 = 0 + 0.7500$ (a-2)

$0.7500 \times 2 = 1 + 0.5000$ (a-3)

$0.5000 \times 2 = 1 + 0.0000$ (a-4)

$\Rightarrow 0.6875_{10} = 0.1011_2$

Binary to hexadecimal:

divide binary number into groups of 4 starting from LSB.

$1000\,1100\,1110_2 = 8CE_{16}$

$0001\,1101\,1011\,1010\,1001 = 1DBA9_{16}$

converse also doable

1010 → A
1011 → B
1100 → C
1101 → D
1110 → E
1111 → F

Binary to Octal:

same as hexadecimal but into groups of 3

$1000\,1100\,1110_2 = 4316_8$

Binary Numbers with fractional part:

$010.101\,100\,101\,100_2 = 2.5454_8$

$0010.101\,100\,101\,100_2 = 2.B2C_{16}$

Hexadecimal to decimal

$1DE4_{16} = 7652_{10} = 4 + 14 \times 16 + 13 \times 16^2 + 1 \times 16^3$

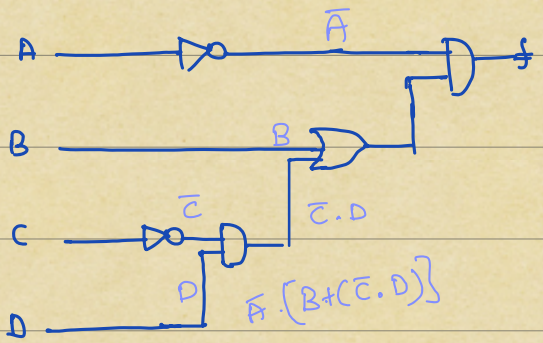
Decimal to Hexadecimal:

	3417
9	213
5	13
13	0

$D59_{16} = 3417_{10}$

Boolean Algebra / Switching Algebra:

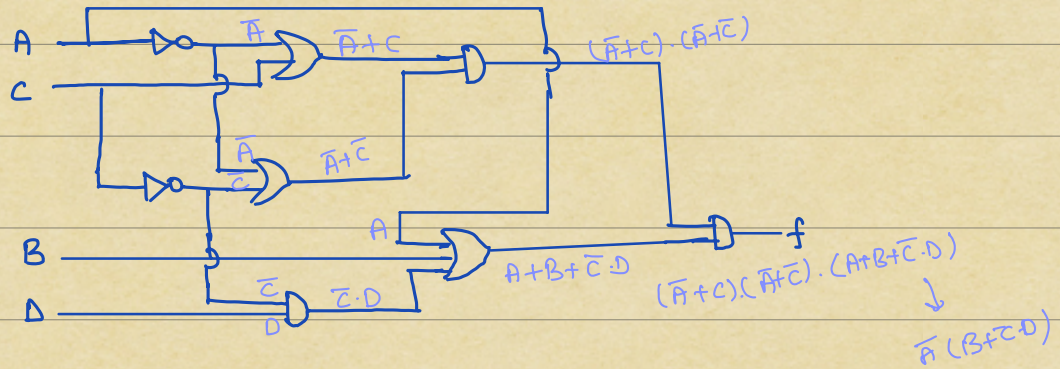
circuit designs



(same inputs, same outputs)

Both are same.

But we prefer the top one as it's simpler



- Boolean algebra and binary logic are related
- key operations: AND, OR, NOT
  (·) (+) (' or −) → complement

- Axioms: → $x+0=x$ $x.1=x$
  → $x+y=y+x$ $x.y=y.x$
  → · is distributive over + $a.(b+c)=a.b+a.c$
  → $x+x'=1$ $x.x'=0$

- Theorms:
  (a) $x+x=x$ (b) $x.x=x$ (c) $x+1=1$ (d) $x.0=0$
  (e) $(x')'=x$ (f) $x+xy=x$ (g) $x.(x+y)=x$
  (h) $(x+y)'=x'.y'$ (i) $(x.y)'=x'+y'$ → demorgan's theorm

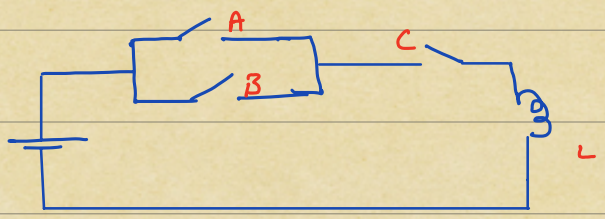
- Duality: To obtain dual of an algebraic expression, simply interchange OR and AND operators replace 1's by 0's and 0's by 1's

- Truth table: way of defining logical operations

x	y	x.y	x+y
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

x	x'
0	1
1	0

→ can be used to prove equivalence of Boolean Exp^.



$L = \bar{A}BC + A\bar{B}C + ABC$

$= C(\bar{A}B + A\bar{B} + AB)$

$= C(A + B)$

A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- Minterms: 2 binary variables can be combined by an AND operation which leads to 4 terms i.e xy', $x'y$, $x'y'$, xy. Each of this term is called an minterm (or standard product)

Suppose 3 terms: ~~xy'~~ ~~zx~~ xyz $yz'x$

all variables must be present

AND(x,y) = min(x,y)

OR(x,y) = max(x,y)

- majority function (only for odd no of inputs)

a	b	c	M(a,b,c)
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$M(a,b,c) = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC + ABC + ABC$

$= BC + CA + AB$

- minterms for n-variables $= 2^n$
- maxterms: $x'+y'$, $x'+y$, $x+y'$, $x+y$ (Standard Sum)

OR(x,y) = max (x,y)

- maxterms for n-variables = 2^n
- $L = A'BC + AB'C + ABC$ → Sum of minterms (sum refers to OR). we may also refer to this as sum of products. an alternate way to express L is as the product (AND) of maxterms.

Canonical form Expressed as sum of minterms or product of maxterms are said to be in canonical forms.

Minterm expansion: a function is written as sum of products. The minterm expansion of a function is unique

maxterm expansion: a function is written as product of maxterms.

We can use minterm expansion to show equivalence of two expressions

$ac' + b'c' + ab = a'b' + bc + ac'$

$(b+b')\ (a+a')\ (c'+c)\ (c+c')\ (a+a')\ (b+b')$

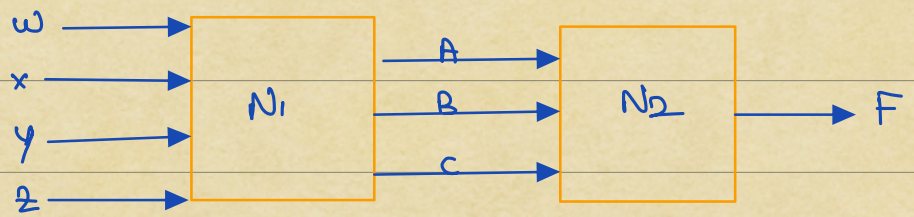
$\Rightarrow abc' + ab'c' + a'b'c' + ab'c' + abc' + abc$

$= a'b'c + a'b'c' + abc + a'bc + abc' + ab'c'$

$\Rightarrow abc + abc' + ab'c' + a'b'c' = abc + abc' + ab'c' + a'b'c'$

$+ a'b'c + a'bc$

Incompletely specified functions:



output of N_1 doesn't generate all possible combinations of ABC

ABC = 001 or 110 → not as inputs to N_2

A	B	C	F
0	0	0	1

X → don't care

0	0	1	X_1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	X_2
1	1	1	1

we need to assign values to both x's to simplify the circuit, and doesn't affect the remaining

$X_1 = X_2 = 0 \Rightarrow F = A'B'C' + BC$

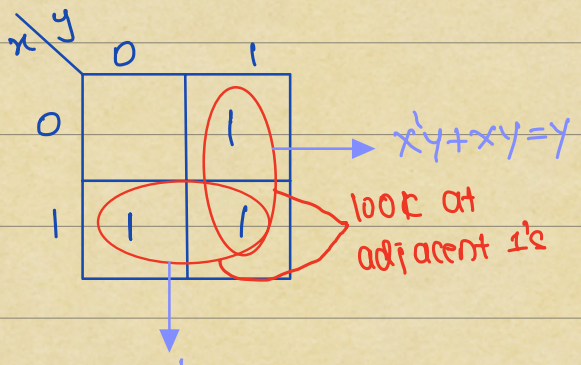
$X_1 = 1, X_2 = 0 \Rightarrow F = A'B' + BC$

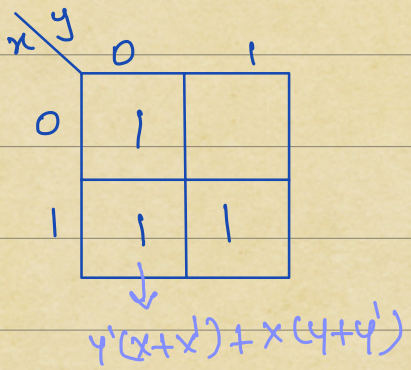
$X_1 = X_2 = 1 \Rightarrow F = AB' + BC + AB$

best choice as it simplifies the circuit more

Karnaugh map: (Veitch diagram)

- goal is to get a simplified form of SOP.
- $xy' + x'y + xy$
- $x + y'$ $\quad xy + xy' + x'y'$



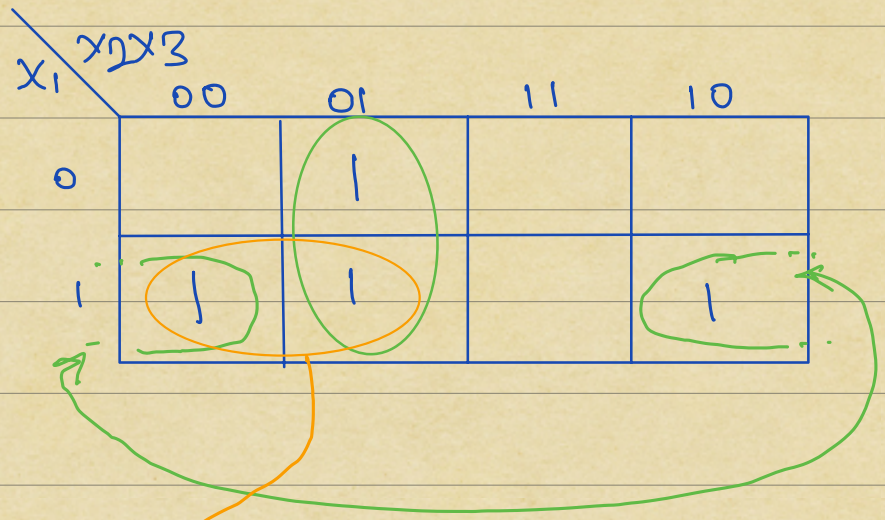


only when 1 variable is changing

only adjacent cells work → else it can't be simplified

k-map for 3 variable functions:

$\bar{x}_1 x_2 x_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 \bar{x}_2 x_3 + x_1 x_2 \bar{x}_3$



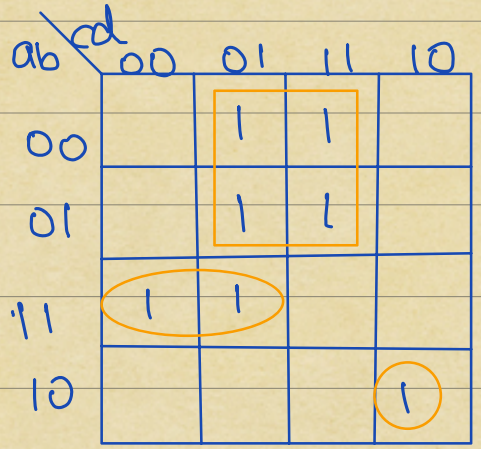
$(\bar{x}_1 \bar{x}_2 x_3 + x_1 \bar{x}_2 x_3) + (x_1 x_2 \bar{x}_3 + x_1 \bar{x}_2 \bar{x}_3)$

$\bar{x}_2 x_3 + x_1 \bar{x}_3$

→ if I consider this loop, we get extra term $x_1 \bar{x}_2$

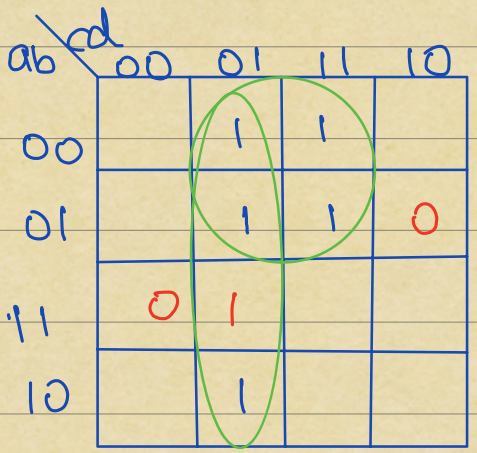
we can get multiple simplified forms for a given truth table

ab \ cd	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10



$\Rightarrow \bar{a}d + ab\bar{c} + a\bar{b}c\bar{d}$

$F = \sum m(1,3,5,7,9) + \sum d(6,12,13)$



$= \bar{c}d + \bar{a}d$

$= d(\bar{a}+\bar{c})$

(best)

Implicants:

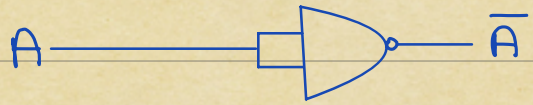
Given a function F of n variables, a product term P is an implicant of F iff for every combination of values of the n variables for which P = 1, F is also equal to 1.

- All of the prime implicants of a function can be obtained from a K-map
- A single 1 represents a prime implicant if its not adjacent to any other 1's.
- Two adjacent 1's form a prime implicant if they aren't contained in a group of four 1's
- Four adjacent 1's form a prime implicant if they aren't contained in a group of eight 1's
- while finding prime implicant don't cares are treated as 1's.

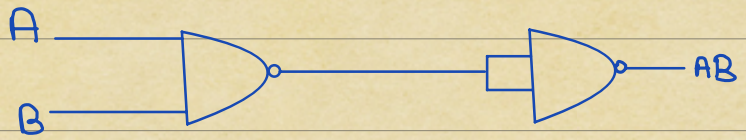
Universal gates:

NAND:

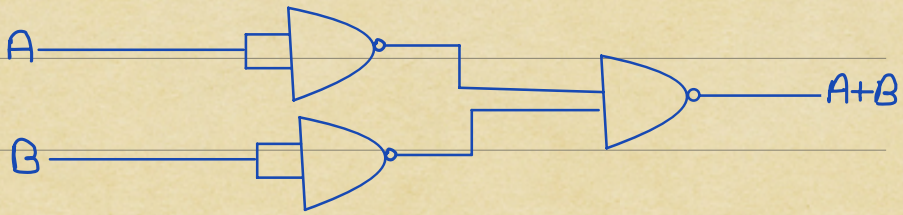
NOT:



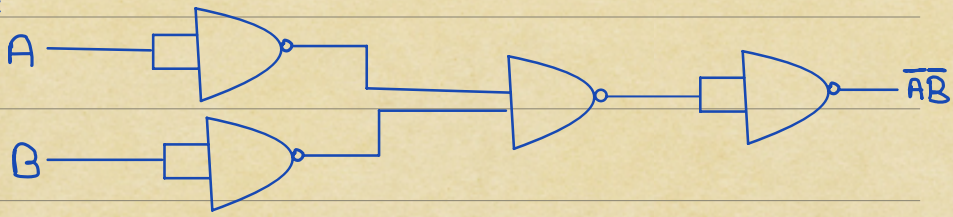
AND:



OR:

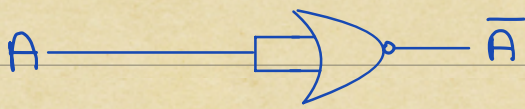


NOR:

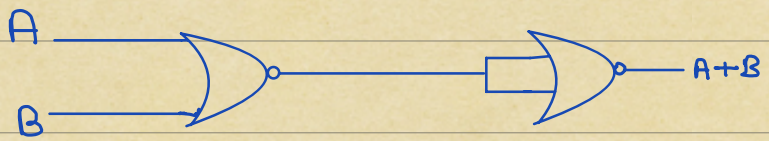


NOR:

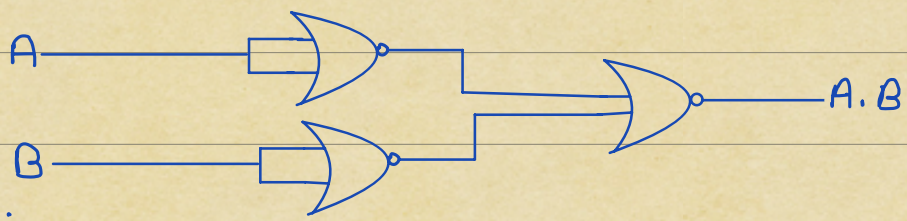
NOT:



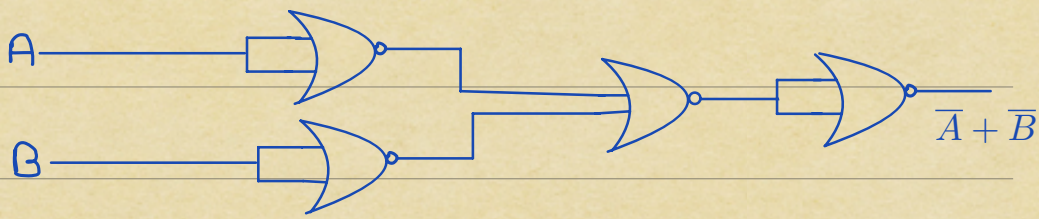
OR:



AND:



NAND:



Combinational circuit: outputs are dependent only on present time input.

Adders: Half adder → A, B → S, C

full adder → A, B, C → S, C

$S = A \oplus B \oplus C$

$C = AB + BC + CA$

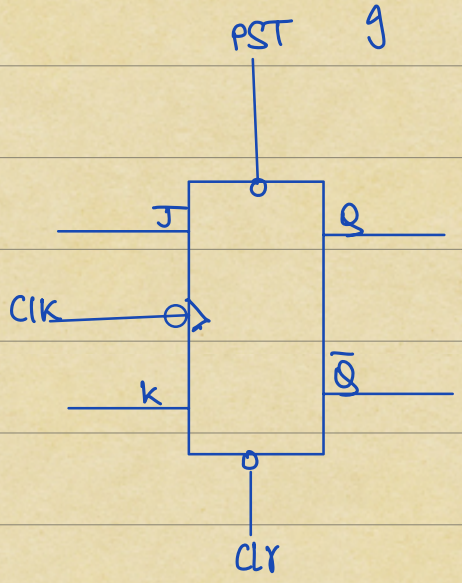
Counters:

n-flip flops $\Rightarrow 2^n$ states $[0$ to $2^n-1]$

Output from Q → up counter

Output from $\bar{Q}$ → down counter

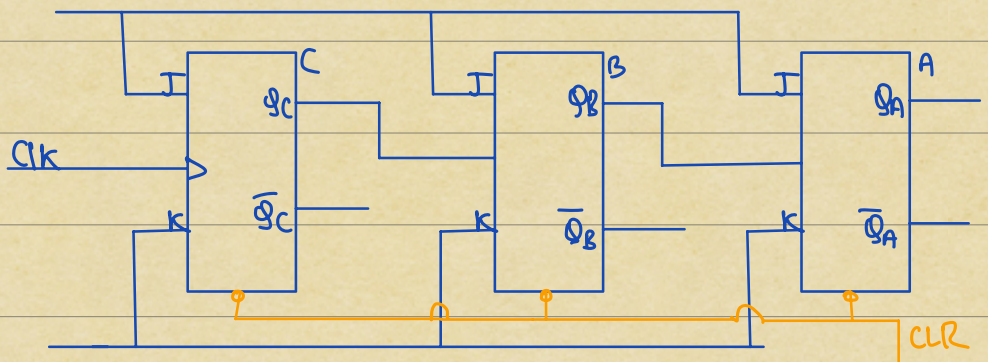
Suppose I want to start/end at a particular state:



An input which activates the circuit when it's low is called active low input

If preset is 0, irrespective of J and K and clock Q becomes 1. Same way if clear is 0 Q becomes 0. These are called overriding inputs

PST	CLR	Q
0	1	1
1	0	0
1	1	follows J,K, clock, acts normally
0	0	not allowed

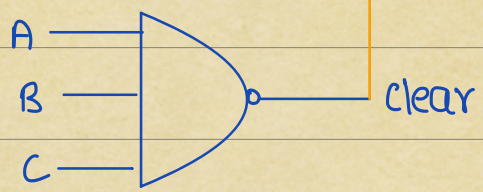


A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

Suppose I want to count 0→5

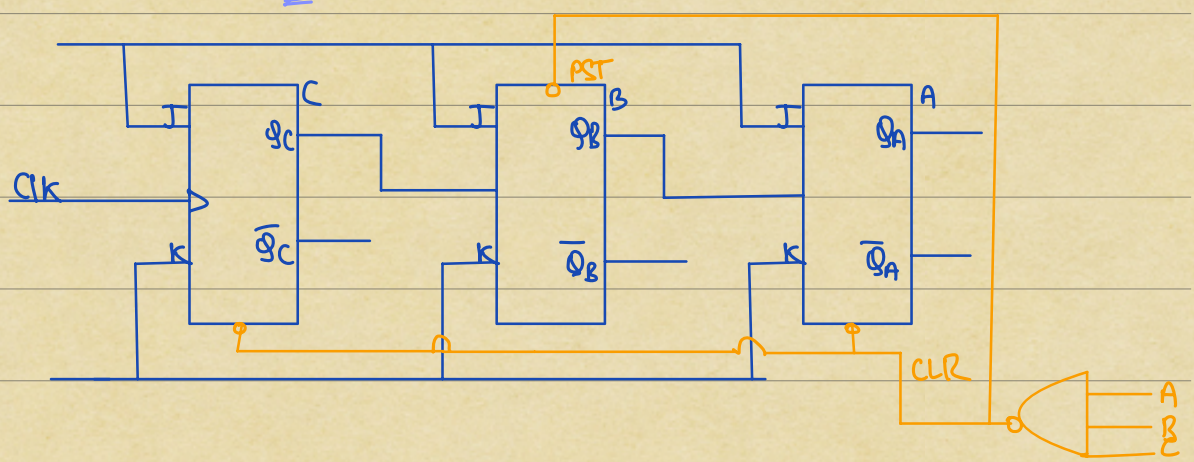
⇒ we wait till state reaches 6 and immediately clear the outputs

A	B	C
1	1	0



Suppose I want to start at count = 2, end at count = 6

2 → 0 1 0 7 → 1 1 1 (2→6)



Question: design a mod-8 synchronous UP/DOWN counter

3 T flip flops

A B C
MSB LSB

Q(n)	Q(n+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

M=0 → up counter
M=1 → down counter

M	Q_A	Q_B	Q_C	Q_A^N	Q_B^N	Q_C^N	T_A	T_B	T_C
0	0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	0	0	1	1
0	0	1	0	0	1	1	0	0	1
0	0	1	1	1	0	0	1	1	1
0	1	0	0	1	0	1	0	0	1
0	1	0	1	1	1	0	0	1	1
0	1	1	0	1	1	1	0	0	1
0	1	1	1	0	0	0	1	1	1
1	0	0	0	1	1	1	1	1	1
1	0	0	1	0	0	0	0	0	1
1	0	1	0	0	0	1	0	1	1

1	0	1	1	0	1	0	0	0	1
1	1	0	0	0	1	1	1	1	1
1	1	0	1	1	0	0	0	0	1
1	1	1	0	1	0	1	0	1	1
1	1	1	1	1	1	0	0	0	1

$T_C = 1$

T_B : mQ_A \ Q_BQ_C

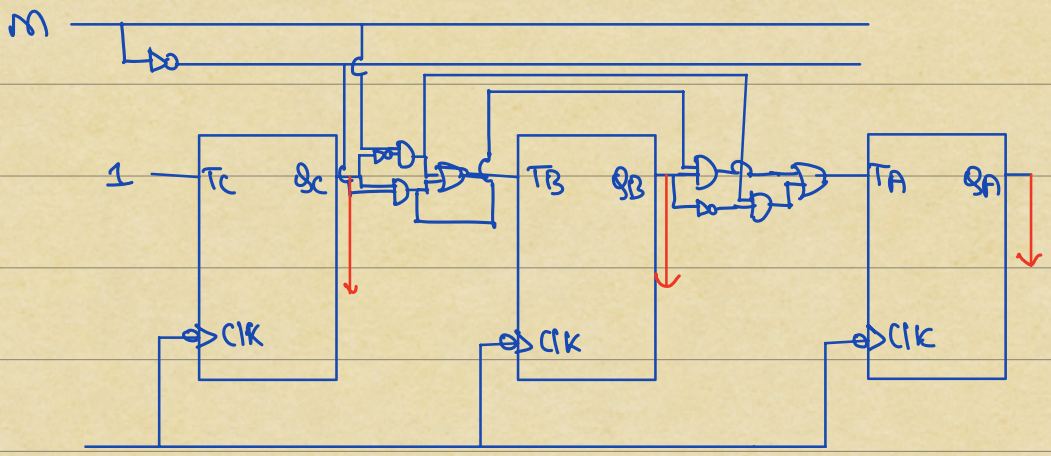
	00	01	11	10
00		1	1	
01		1	1	
11	1			1
10	1			1

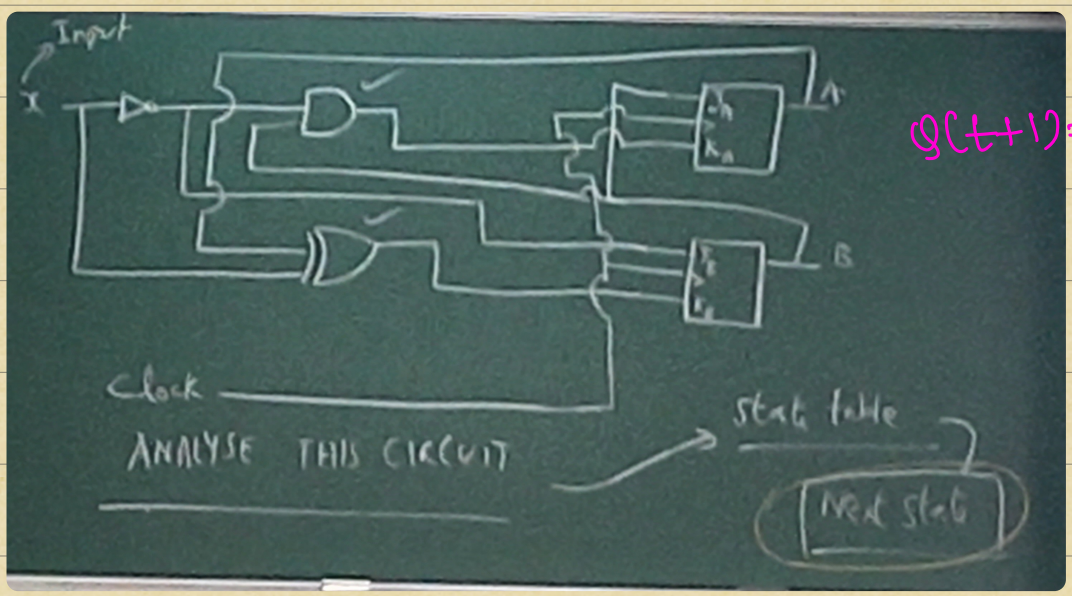
$= \bar{m}Q_C + m\bar{Q}_C$

T_A : mQ_A \ Q_BQ_C

		1	
		1	
1			
1			

$= \bar{m}Q_BQ_C + m\bar{Q}_B\bar{Q}_C$





$Q(t+1) = J\bar{Q} + \bar{K}Q$

$J_A = B \quad J_B = \bar{x} \quad K_A = B\bar{x} \quad K_B = A \oplus x$

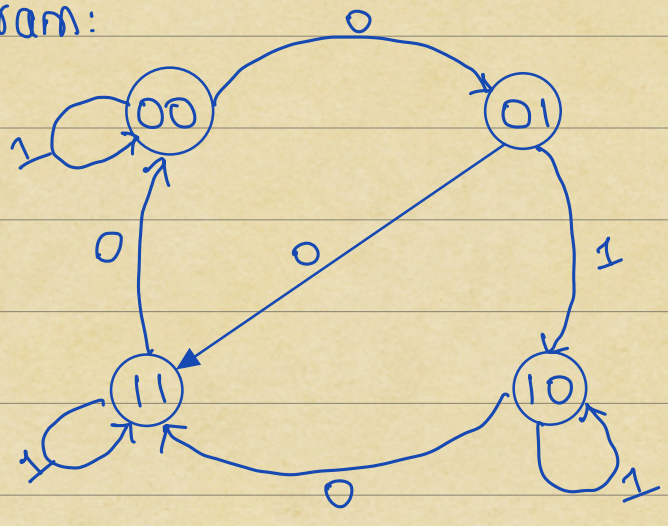
$A(t+1) = B\bar{A} + \overline{B\bar{x}}A \quad B(t+1) = \bar{x}\bar{B} + (A \odot x)B$

State table:

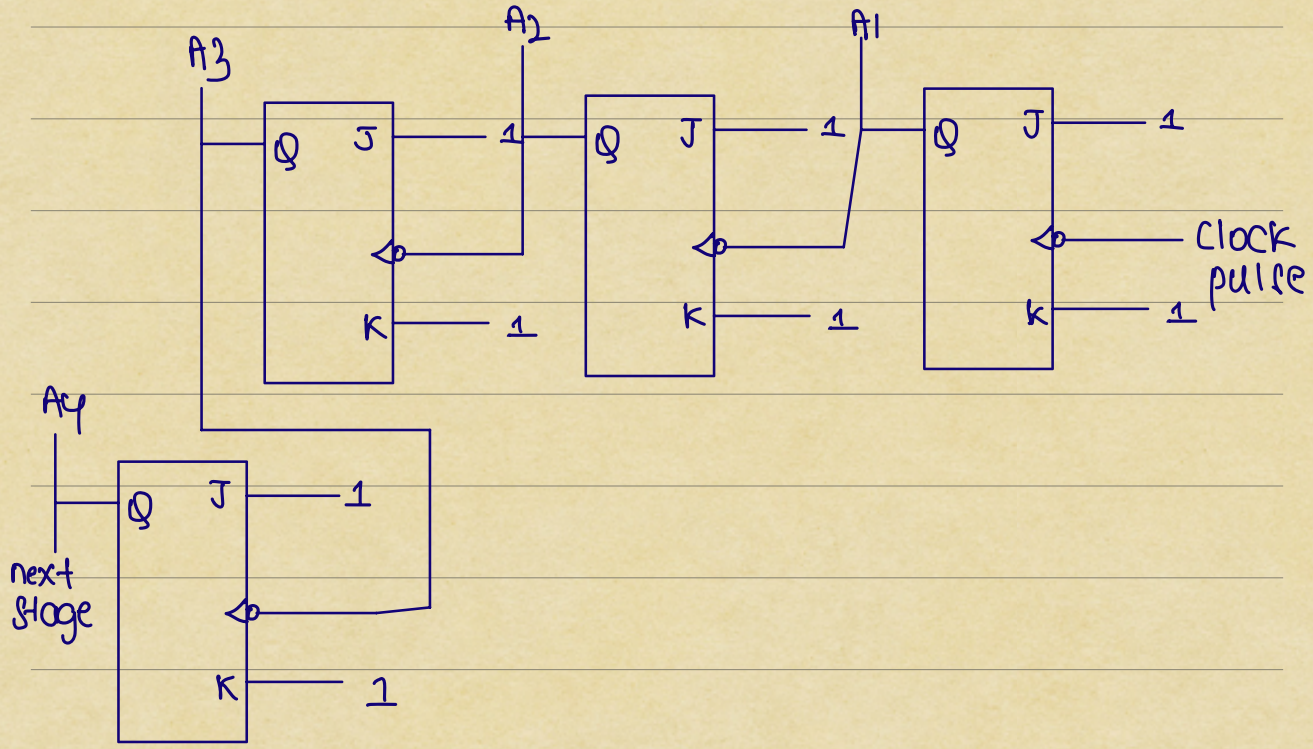
present		input	Next	
A	B	x	A	B
0	0	0	0	1
0	0	1	0	0
0	1	0	1	1

0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	1	1

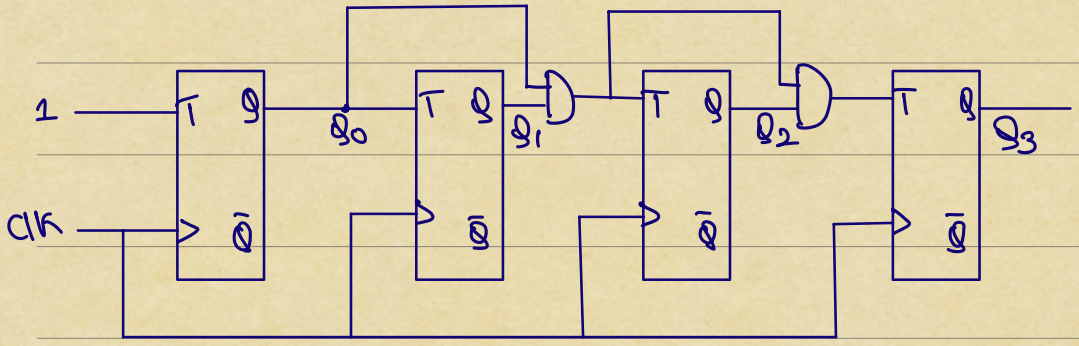
State diagram:



Ripple counter



Synchronous counter: (modulo 16 counter)



$T_0 = 1 \quad T_1 = Q_0 \quad T_2 = Q_0 Q_1 \quad T_3 = Q_0 Q_1 Q_2$

Basic Comparator:

